Övning – Arv och interface

1 Arv, Shapes

a) Skriv en klass, Shape, som ska utgöra en superklass för ritbara former.

Klassen ska ha privata datamedlemmar för position, två heltal x och y.
Det ska finnas 2 konstruktorer, en utan argument och en som tar utgångspositionen (x och y) som argument.
Det ska även finnas metoder för att sätta respektive avläsa x och y (”setters” och ”getters”) samt en omdefinierad toString.

Skriv sedan en klass, Circle, som ärver Shape (extends). Klassen Circle ska ha en privat datamedlem som representerar cirkelns radie, två konstruktorer, getters och setters för radien samt en omdefinierad toString-metod. Metoden toString ska reutrnera en text som även visar det ärvda datat, x och y.

Skapa en separat klass med en main-metod och skapa där ett Shape-objekt samt ett Circle-objekt. Vilka metoder kan du anropå på CIrcle-objektet? Testa!

Skapa också en ny Shape-referens och låt den referera cirkel-objektet, se nedan.
Vilken version av metoden toString förväntar du dig ska exekveras vid anrop från shape-referensen på circle-objektet? Varför?

```
Circle c = new Circle(20,20);
…
Shape cs = c; // cast av referensen
System.out.printn(cs.toString());  // ?
```

b) Skapa klassen Rectangle, som ärver av Shape. Lägg till privata datamedlemmar, x2 och y2, för rektangelns andra hörn.
Lägg också till två konstruktorer, getters och setters för x2 och y2 samt en omdefinierad toString.

Skapa, i main-metoden, en array med Shape-referenser och låt dessa referera objekt av de tre typerna i arvshierarkin. Vilka metoder kan du anropa via shape-referenserna? Testa.

För att anropa en metod specifik för t.ex. Circle, måste referensen, och objektet, vara av just typen Circle. Gör en ”downcast” av referensen för ett crkelobjekt i din array och testa. Ex:

```
Circle c = (Circle) shapes[42]; // downcast av referensen
int r = c.getRadius(); // nu möjligt
```

2. Abstrakt superklass

Gör klassen Shape abstrakt – dvs. märk klassen ”abstract”. Vad innebär detta för klassen? (Ledning: vi vill ju egentligen inte använda oss av shape-objekt utan endast av subtyperna.)

Lägg till en abstrakt metod, draw, i Shape. Metoden är tänkt definiera hur en speciell form ska ritas ut på skärmen.

Metoden deklareras
”public abstract void draw(); ” – notera att metoden inte har någon implementation.

När du sparar filen får du felmarkeringar på sub-klasserna Varför?

Implementera metoden draw i sub-klasserna. Eftersom vi inte orkar skriva grafisk kod fuskar vi och anropa toString i draw och skriver informationen till terminalen (använd referensen this).

Skriv en main-metod där du skapar shape-referenser som refererar objekt av sub-typerna och anropar draw för dessa.
Notera att du skrivit polymorf kod när du anropar draw på detta sätt.

3. Java-interface

Skriv ett interface, Movable, som deklarerar en metod, move, för att förflytta något ett antal steg. Metoden ska ha två argument, dx och dy.

Markera att klassen Shape implementerar detta interface via ”implements Movable”.
Normalt skulle vi nu tvingas implementera metoden move i klassen Shape, men i eftersom Shape är abstrakt behöver vi inte det. Däremot måste vi implementera metoden move i sub-klasserna eftersom dessa ärver egenskapen Movable från Shape.

Låt Circle implementera move så att endast x och y flyttas, medan vi i Rectangles move flyttar både x, y, x2 och y2.

Skriv en main-metod med en array av Movable-referenser och låt dessa referera till Shape-objekt (tillåtet eftersom Shape implementerar Movable).
Vilka metoder kan anropas via Movable referenserna? Gör detta och kontrollera att allt funfgerar som förväntat.

[Notera att vi kunde ha lagt move, utan interface, direkt i super-klassen Shape; men vi ville ju testa interface…]

4. java.lang.Comparable

För att det ska vara möjligt att jämföra kort, till exempel sortera en hand, är det lämpligt att låta klassen Card implementera interfacet `Comparable<T>`, som har till uppgift att definiera den naturliga rangordningen för ett objekt av en viss klass. Sök upp detta interface i API-dokumentationen och läs om metoden `compareTo(T other)`.

På kurswebben finns en klass `Card` att utgå ifrån. Låt din kortklass implementera interfacet `Comparable` så att ordningen avgöras efter färg, och om färgerna är lika, efter valör. Använd enum-metoden `ordinal`, som returnerar ett heltal som representerar ordningen i denna enum, till exempel `rank.ordinal()`.

Skriv ett main där du skapar ett antal kort i en lista, osorterad, och sedan använder dig av klassmetoden `sort(…)` i klassen `java.utils.Collections` för att sortera dessa – denna metod använder sig av implementationen av `Comparable` vid jämförelser.

Om en klass implementerar `Comparable` rekommenderas även att metoden `equals`, ärvd från `Object`, omdefinieras så att `(c1.compareTo(c2) == 0)` ⇔ `(c1.equals(c2) == true)`. Lägg till denna metod i kortklassen.

5. Money med Comparable<T>

Du ska skriva en klass, Money, vars objekt representerar ett belopp. Förutom beloppet har en Money-instans en valuta, som representeras av en enum Currency, se nedan (du behöver inte skriva Currency själv).

Klassen Money specificeras av interfacet IMoney, se nedan. Din lösning ska implementera detta interface som även utökar interfacet java.lang.Comparable. Objekt av typen Money ska vara immutable, men det ska finnas metoder som skapar nya objekt, utifrån ett existerande objekt, med nya egenskaper.

Förutom de metoder som definieras i interfacet ska din klass ha tre konstruktorer, se dokumentationen för interfacet nedan.

enum Currency

```
public enum Currency {
    EUR("euro", 1.00),
    SEK("kronor", 0.095),
    USD("dollar", 0.91),
    NOK("kronor", 0.092);

    public String getName() {
        return name;
    }

    public double getRate() {
        return rate;
    }

    private String name;
    private double rate; // rate vs EUR

    private Currency(String name, double rate) {
        this.name = name;
        this.rate = rate;
    }
}
```

interface IMoney

```
/**
 * Represents an amount of money, represented in a specific currency.
 * Instances of this class are <em>immutable</em>.
 * <p>
 * The implementing class must have three constructors, with two arguments,
 * (throws IllegalArgumentException if amount < 0) one argument (amount 0),
 * and no arguments (amount 0, currency EUR).
 */
```

```
public interface IMoney extends Comparable<T> {

    /**
     * Return the amount for this Money.
     */
    double getAmount();

    /**
     * Return the currency for this Money.
     */
    Currency getCurrency();

    /**
     * Return a copy of this instance.
     */
    Money copy();

    /**
     * Create a new Money instance, with the indicated currency and the
     *
     * @param newCurrency
     * @return a new Money instance.
     */
    Money toCurrency(Currency newCurrency);

    /**
     * Create a new Money instance with the sum, in the same currency as this.
     *
     * @param moneyToAdd the money to add to this money
     * @return a new Money instance in the same currency as this.
     */
    Money add(Money moneyToAdd);

    /**
     * Create a new Money instance with the difference, in the same currency as this.
     *
     * @param moneyToAdd the money to add to this money
     * @return a new Money instance in the same currency as this.
     * @throws IllegalArgumentException if the sum is less than zero.
     */
    Money subtract(Money moneyToAdd) throws IllegalArgumentException;

    /**
     * Return a string with the amount, rounded to two decimals, and the currency.
     */
    @Override
    String toString();

    /**
     * Compare amounts, internally converted to EUR. Two instances are
     * considered equal if amounts differ less than 1E-3.
     * Also see documentation for interface java.lang.Comparable.
     */
    @Override
    int compareTo(Money other);

    /**
     * Compare amounts, internally converted to EUR. Two instances are
     * considered equal if amounts differ less than 1E-3.
     * Also see documentation for class java.lang.Object.
     */
    @Override
    boolean equals(Object other);
}
```

6. java.lang.Comparator<T>

Interfacet `Comparator<T>` följer designmönstret Strategy och gör det möjligt att definiera flera olika sätt att rangordna objekt av en viss typ. Läs om `java.util.Comparable` i javas API-dokumentation - notera att interfacet, precis som Comparable, är generiskt, med en typparameter T.

Skriv två implementationer av interfacet, och metoden `compare(T obj1, T obj2)`. Den första ska rangordna enbart efter rank, den andra ska rangordna efter suit men, om suit är lika, efter rank och då räkna ACE som det högsta värdet.

Skriv sedan en main-metod där du skapar `Card`-objekt i en lista och sorterar dessa med metoden `java.util.Collections.sort(list, comparator)` – du skickar både listan och comparator-implementationen till sorteringsmetoden.